arXiv:1608.05859v3 [cs.CL] 21 Feb 2017

Using the Output Embedding to Improve Language Models

Ofir Press and **Lior Wolf**
School of Computer Science
Tel-Aviv University, Israel
{ofir.press,wolf}@cs.tau.ac.il

Abstract

We study the topmost weight matrix of neural network language models. We show that this matrix constitutes a valid word embedding. When training language models, we recommend tying the input embedding and this output embedding. We analyze the resulting update rules and show that the tied embedding evolves in a more similar way to the output embedding than to the input embedding in the untied model. We also offer a new method of regularizing the output embedding. Our methods lead to a significant reduction in perplexity, as we are able to show on a variety of neural network language models. Finally, we show that weight tying can reduce the size of neural translation models to less than half of their original size without harming their performance.

1 Introduction

In a common family of neural network language models, the current input word is represented as the vector $c \in \mathbb{R}^C$ and is projected to a dense representation using a word embedding matrix U. Some computation is then performed on the word embedding $U^\top c$, which results in a vector of activations h_2. A second matrix V then projects h_2 to a vector h_3 containing one score per vocabulary word: $h_3 = Vh_2$. The vector of scores is then converted to a vector of probability values p, which represents the models' prediction of the next word, using the softmax function.

For example, in the LSTM-based language models of (Sundermeyer et al., 2012; Zaremba et al., 2014), for vocabulary of size C, the one-hot encoding is used to represent the input c and $U \in \mathbb{R}^{C\times H}$. An LSTM is then employed, which results in an activation vector h_2 that similarly to $U^\top c$, is also in $\mathbb{R}^H$. In this case, U and V are of exactly the same size.

We call U the input embedding, and V the output embedding. In both matrices, we expect rows that correspond to similar words to be similar: for the input embedding, we would like the network to react similarly to synonyms, while in the output embedding, we would like the scores of words that are interchangeable to be similar (Mnih and Teh, 2012).

While U and V can both serve as word embeddings, in the literature, only the former serves this role. In this paper, we compare the quality of the input embedding to that of the output embedding, and we show that the latter can be used to improve neural network language models. Our main results are as follows: (i) We show that in the word2vec skip-gram model, the output embedding is only slightly inferior to the input embedding. This is shown using metrics that are commonly used in order to measure embedding quality. (ii) In recurrent neural network based language models, the output embedding outperforms the input embedding. (iii) By tying the two embeddings together, i.e., enforcing $U = V$, the joint embedding evolves in a more similar way to the output embedding than to the input embedding of the untied model. (iv) Tying the input and output embeddings leads to an improvement in the perplexity of various language models. This is true both when using dropout or when not using it. (v) When not using dropout, we propose adding an additional projection P before V, and apply regularization to P. (vi) Weight tying in neural translation models can reduce their size (number of parameters) to less than half of their original size without harming their performance.

arXiv:1608.05859v3 [cs.CL] 21 Feb 2017

利用输出嵌入优化语言模型

以色列特拉维夫大学利奥尔·沃尔夫
计算机科学学院与Ofir Press{
ofir.press,wolf}@cs.tau.ac.il

摘要

我们研究了神经网络语言模型中最上层的权重矩阵，并证明该矩阵实际上构成了一种有效的词嵌入。在训练语言模型时，我们建议将输入嵌入与这种输出嵌入进行权重绑定。通过分析由此产生的更新规则，我们发现经过绑定的嵌入其演化方式更接近输出嵌入，而非未绑定模型中的输入嵌入。此外，我们还提出了一种用于对输出嵌入进行正则化的新方法。正如我们在多种神经网络语言模型上所展示的那样，这些方法能够显著降低模型的困惑度。最后，我们证明权重绑定可以将神经网络翻译模型的规模缩小到原来的一半以下，同时不会影响其性能。

1 引言

在常见的神经网络语言模型体系中，当前的输入词会被表示为向量 $c \in \mathrm{IR}^C$ ，随后通过词嵌入矩阵 U被投影为密集表示。接着会对该词嵌入 $U^\top c$进行一些运算，从而得到一个激活向量 h_2。之后另一个矩阵 V会将 h_2投影为向量 h_3 ，该向量中包含每个词汇表单词对应的得分： $h_3 = Vh_2$。最后，这个得分向量会通过softmax函数转换为概率值向量 p，以此体现模型对下一个单词的预测结果。

例如，在基于LSTM网络的语言模型中（如Sundermeyer等人，2012年；Zaremba等人，2014年所提出的模型），对于规模为 C的词汇表，会采用独热编码的方式来表示输入c 以及 $U \in \mathrm{IR}^{C\times H}$。随后会使用LSTM网络来处理这些数据。

这样一来就会生成一个激活向量 h_2 ，该向量与 $U^\top c$类似，同样属于IR^H范畴。在这种情况下， U和 V的规模是完全相同的。

我们将 U 称为输入嵌入，而 V 称为输出嵌入。在这两个矩阵中，我们期望表示相似单词的行也具有相似性：对于输入嵌入，我们希望网络对同义词产生类似的反应；而对于输出嵌入，我们则希望可以互相替代的单词对应的得分也保持相似（Mnih和Teh，2012）。

While U and V can both serve as word embeddings, in the literature, only the former serves this role. In this paper, we compare the quality of the input embedding to that of the output embedding, and we show that the latter can be used to improve neural network language models. Our main results are as follows: (i) We show that in the word2vec skip-gram model, the output embedding is only slightly inferior to the input embedding. This is shown using metrics that are commonly used in order to measure embedding quality. (ii) In recurrent neural network based language models, the output embedding outperforms the input embedding. (iii) By tying the two embeddings together, i.e., enforcing $U = V$, the joint embedding evolves in a more similar way to the output embedding than to the input embedding of the untied model. (iv) Tying the input and output embeddings leads to an improvement in the perplexity of various language models. This is true both when using dropout or when not using it. (v) When not using dropout, we propose adding an additional projection P before V, and apply regularization to P. (vi) Weight tying in neural translation models can reduce their size (number of parameters) to less than half of their original size without harming their performance.

2 Related Work

Neural network language models (NNLMs) assign probabilities to word sequences. Their resurgence was initiated by (Bengio et al., 2003). Recurrent neural networks were first used for language modeling in (Mikolov et al., 2010) and (Pascanu et al., 2013). The first model that implemented language modeling with LSTMs (Hochreiter and Schmidhuber, 1997) was (Sundermeyer et al., 2012). Following that, (Zaremba et al., 2014) introduced a dropout (Srivastava, 2013) augmented NNLM. (Gal, 2015; Gal and Ghahramani, 2016) proposed a new dropout method, which is referred to as Bayesian Dropout below, that improves on the results of (Zaremba et al., 2014).

The skip-gram word2vec model introduced in (Mikolov et al., 2013a; Mikolov et al., 2013b) learns representations of words. This model learns a representation for each word in its vocabulary, both in an input embedding matrix and in an output embedding matrix. When training is complete, the vectors that are returned are the input embeddings. The output embedding is typically ignored, although (Mitra et al., 2016; Mnih and Kavukcuoglu, 2013) use both the output and input embeddings of words in order to compute word similarity. Recently, (Goldberg and Levy, 2014) argued that the output embedding of the word2vec skip-gram model needs to be different than the input embedding.

As we show, tying the input and the output embeddings is indeed detrimental in word2vec. However, it improves performance in NNLMs.

In neural machine translation (NMT) models (Kalchbrenner and Blunsom, 2013; Cho et al., 2014; Sutskever et al., 2014; Bahdanau et al., 2014), the decoder, which generates the translation of the input sentence in the target language, is a language model that is conditioned on both the previous words of the output sentence and on the source sentence. State of the art results in NMT have recently been achieved by systems that segment the source and target words into subword units (Sennrich et al., 2016a). One such method (Sennrich et al., 2016b) is based on the byte pair encoding (BPE) compression algorithm (Gage, 1994). BPE segments rare words into their more commonly appearing subwords.

Weight tying was previously used in the log-bilinear model of (Mnih and Hinton, 2009), but the decision to use it was not explained, and its effect on the model's performance was not tested. Independently and concurrently with our work (Inan et al., 2016) presented an explanation for weight tying in NNLMs based on (Hinton et al., 2015).

3 Weight Tying

In this work, we employ three different model categories: NNLMs, the word2vec skip-gram model, and NMT models. Weight tying is applied similarly in all models. For translation models, we also present a three-way weight tying method.

NNLM models contain an input embedding matrix, two LSTM layers (h_1 and h_2), a third hidden scores/logits layer h_3, and a softmax layer. The loss used during training is the cross entropy loss without any regularization terms.

Following (Zaremba et al., 2014), we employ two models: large and small. The large model employs dropout for regularization. The small model is not regularized. Therefore, we propose the following regularization scheme. A projection matrix $P \in \mathbb{R}^{H \times H}$ is inserted before the output embedding, i.e., $h_3 = VPh_2$. The regularizing term $\lambda \|P\|_2$ is then added to the small model's loss function. In all of our experiments, $\lambda = 0.15$.

Projection regularization allows us to use the same embedding (as both the input/output embedding) with some adaptation that is under regularization. It is, therefore, especially suited for WT.

While training a vanilla **untied NNLM**, at timestep t, with current input word sequence $i_{1:t} = [i_1, ..., i_t]$ and current target output word o_t, the negative log likelihood loss is given by: $\mathcal{L}_t = -\log p_t(o_t|i_{1:t})$, where $p_t(o_t|i_{1:t}) = \frac{\exp(V_{o_t}^\top h_2^{(t)})}{\sum_{x=1}^{C} \exp(V_x^\top h_2^{(t)})}$, U_k (V_k) is the kth row of U (V), which corresponds to word k, and $h_2^{(t)}$ is the vector of activations of the topmost LSTM layer's output at time t. For simplicity, we assume that at each timestep t, $i_t \neq o_t$. Optimization of the model is performed using stochastic gradient descent.

The update for row k of the input embedding is:

$$\frac{\partial \mathcal{L}_t}{\partial U_k} = \begin{cases} (\sum_{x=1}^{C} p_t(x|i_{1:t}) \cdot V_x^\top - V_{o_t}^\top) \frac{\partial h_2^{(t)}}{\partial U_{i_t}} & k = i_t \\ 0 & k \neq i_t \end{cases}$$

For the output embedding, row k's update is:

$$\frac{\partial \mathcal{L}_t}{\partial V_k} = \begin{cases} (p_t(o_t|i_{1:t}) - 1) h_2^{(t)} & k = o_t \\ p_t(k|i_{1:t}) \cdot h_2^{(t)} & k \neq o_t \end{cases}$$

2 相关工作

Neural network language models (NNLMs) assign probabilities to word sequences. Their resurgence was initiated by (Bengio et al., 2003). Recurrent neural networks were first used for language modeling in (Mikolov et al., 2010) and (Pascanu et al., 2013). The first model that implemented language modeling with LSTMs (Hochreiter and Schmidhuber, 1997) was (Sundermeyer et al., 2012). Following that, (Zaremba et al., 2014) introduced a dropout (Srivastava, 2013) augmented NNLM. (Gal, 2015; Gal and Ghahramani, 2016) proposed a new dropout method, which is referred to as Bayesian Dropout below, that improves on the results of (Zaremba et al., 2014).

在（Mikolov等人，2013a；Mikolov等人，2013b）中提出的skip-gram word2vec模型可用于学习单词的表示形式。该模型会为其词汇表中的每个单词分别在输入嵌入矩阵和输出嵌入矩阵中生成对应的表示向量。训练完成后，返回的向量即为输入嵌入向量，而输出嵌入向量通常会被忽略，尽管（Mitra等人，2016；Mnih和Kavukcuoglu，2013）曾同时利用单词的输入嵌入和输出嵌入来计算单词相似度。近期，（Goldberg和Levy，2014）提出，word2vec skip-gram模型的输出嵌入应与输入嵌入有所不同。

正如我们所展示的，在word2vec模型中将输入嵌入与输出嵌入绑定在一起确实会带来负面影响，但在神经网络语言模型中却能提升性能。

在神经机器翻译（NMT）模型中（Kalchbrenner和Blunsom，2013；Cho等人，2014；Sutskever等人，2014；Bahdanau等人，2014），用于生成目标语言译文的解码器实际上是一种语言模型，它会同时参考输出句子中的前序单词以及源句子的内容。近期，通过将源语言和目标语言的单词分割为子词单元，相关系统在NMT领域取得了最先进的成果（Sennrich等人，2016a）。其中一种方法（Sennrich等人，2016b）基于字节对编码（BPE）压缩算法（Gage，1994）实现，该算法会将那些出现频率较低的单词拆分成更常见的子词。

权重绑定此前曾被用于对数运算中。

在（Mnih和Hinton于2009年提出的）对数双线性模型中，此前曾使用过权重绑定技术，但当时并未说明采用该技术的理由，也未对其对模型性能的影响进行过测试。与我们的研究（Inan等人，2016年）几乎同时，另有研究基于（Hinton等人，2015年的成果）为神经网络语言模型中的权重绑定提供了相应的解释。

3 权重绑定

在本次研究中，我们采用了三类不同的模型：神经网络语言模型、word2vec跳过语法模型以及机器翻译模型。所有模型均应用了权重绑定技术。针对翻译模型，我们还提出了一种三向权重绑定方法。

NNLM模型包含一个输入嵌入矩阵、两个LSTM网络（h_1 和 h_2）、第三个用于生成隐藏分数/逻辑值的层 h_3，以及一个softmax层。在训练过程中所使用的损失函数为不包含任何正则化项的交叉熵损失。

遵循Zaremba等人，2014年的研究方法，我们采用了两种规模的模型：大型模型与小型模型。大型模型会运用丢弃机制来实现正则化，而小型模型则不进行任何正则化处理。因此我们提出了如下的正则化方案：在输出嵌入即 $h_3 = VPh_2$之前插入一个投影矩阵$P \in \mathrm{IR}^{H \times H}$，随后再将正则化项$\lambda\|P\|_2$ 添加到小型模型的损失函数中。在所有的实验中，$\lambda = 0.15$。

通过投影正则化，我们可以使用相同的嵌入结构（既作为输入嵌入也作为输出嵌入），同时通过对该嵌入进行特定的调整并施加正则化约束。正因如此，这种方法尤其适用于加权文本任务。

在训练普通的未加权神经网络语言模型时，在时间步 t，面对当前的输入词序列 $i_{1:t} = [i_1, ..., i_t]$ 和对应的目标输出词o_t，负对数似然损失的计算公式为：$\mathcal{L}_t = -\log p_t(o_t|i_{1:t})$，其中 $p_t(o_t|i_{1:t}) = \frac{\exp(V_{o_t}^\top h_2^{(t)})}{\sum_{x=1}^{C} \exp(V_x^\top h_2^{(t)})}$，$U_k(V_k)$是 $U(V)$的第 k行，该行对应的是词 k；而 $h_2^{(t)}$ 则是时间 t时最顶层LSTM层输出对应的激活值向量。为简化问题，我们假设在每个时间步 t时 $i_t \neq o_t$均成立。该模型的优化是通过随机梯度下降法来实现的。

输入嵌入中第 k 行的更新值为：

$$\frac{\partial \mathcal{L}_t}{\partial U_k} = \begin{cases} (\sum_{x=1}^{C} p_t(x|i_{1:t}) \cdot V_x^\top - V_{o_t}^\top) \frac{\partial h_2^{(t)}}{\partial U_{i_t}} & k = i_t \\ 0 & k \neq i_t \end{cases}$$

对于输出词嵌入而言，第 k行的更新内容为：

$$\frac{\partial \mathcal{L}_t}{\partial V_k} = \begin{cases} (p_t(o_t|i_{1:t}) - 1) h_2^{(t)} & k = o_t \\ p_t(k|i_{1:t}) \cdot h_2^{(t)} & k \neq o_t \end{cases}$$

Therefore, in the untied model, at every timestep, the only row that is updated in the input embedding is the row U_{i_t} representing the current input word. This means that vectors representing rare words are updated only a small number of times. The output embedding updates every row at each timestep.

In **tied NNLMs**, we set $U = V = S$. The update for each row in S is the sum of the updates obtained for the two roles of S as both an input and output embedding.

The update for row $k \neq i_t$ is similar to the update of row k in the untied NNLM's output embedding (the only difference being that U and V are both replaced by a single matrix S). In this case, there is no update from the input embedding role of S.

The update for row $k = i_t$, is made up of a term from the input embedding (case $k = i_t$) and a term from the output embedding (case $k \neq o_t$). The second term grows linearly with $p_t(i_t|i_{1:t})$, which is expected to be close to zero, since words seldom appear twice in a row (the low probability in the network was also verified experimentally). The update that occurs in this case is, therefore, mostly impacted by the update from the input embedding role of S.

To conclude, in the tied NNLM, every row of S is updated during each iteration, and for all rows except one, this update is similar to the update of the output embedding of the untied model. This implies a greater degree of similarity of the tied embedding to the untied model's output embedding than to its input embedding.

The analysis above focuses on NNLMs for brevity. In **word2vec**, the update rules are similar, just that $h_2^{(t)}$ is replaced by the identity function. As argued by (Goldberg and Levy, 2014), in this case weight tying is not appropriate, because if $p_t(i_t|i_{1:t})$ is close to zero then so is the norm of the embedding of i_t. This argument does not hold for NNLMs, since the LSTM layers cause a decoupling of the input and output embedddings.

Finally, we evaluate the effect of weight tying in **neural translation models**. In this model: $p_t(o_t|i_{1:t}, r) = \frac{exp(V_{o_t}^\top G^{(t)})}{\sum_{x=1}^{C_t} exp(V_x^\top G^{(t)})}$ where $r = (r_1, ..., r_N)$ is the set of words in the source sentence, U and V are the input and output embeddings of the decoder and W is the input embedding of the encoder (in translation models $U, V \in \mathbb{R}^{C_t \times H}$ and $W \in \mathbb{R}^{C_s \times H}$, where C_s / C_t is the size of the vocabulary of the source / target). $G^{(t)}$ is the decoder, which receives the context vector, the embedding of the input word (i_t) in U, and its previous state at each timestep. c_t is the context vector at timestep t, $c_t = \sum_{j \in r} a_{tj} h_j$, where a_{tj} is the weight given to the jth annotation at time t: $a_{tj} = \frac{\exp(e_{tj})}{\sum_{k \in r} \exp(e_{ik})}$, and $e_{tj} = a_t(h_j)$, where a is the alignment model. F is the encoder which produces the sequence of annotations $(h_1, ..., h_N)$.

The output of the decoder is then projected to a vector of scores using the output embedding: $l_t = VG^{(t)}$. The scores are then converted to probability values using the softmax function.

In our weight tied translation model, we tie the input and output embeddings of the decoder.

We observed that when preprocessing the ACL WMT 2014 EN→FR[1] and WMT 2015 EN→DE[2] datasets using BPE, many of the subwords appeared in the vocabulary of both the source and the target languages. Tab. 1 shows that up to 90% (85%) of BPE subwords between English and French (German) are shared.

Language pairs	Subwords only in source	Subwords only in target	Subwords in both
EN→FR	2K	7K	85K
EN→DE	3K	11K	80K

Table 1: Shared BPE subwords between pairs of languages.

Based on this observation, we propose three-way weight tying (TWWT), where the input embedding of the decoder, the output embedding of the decoder and the input embedding of the encoder are all tied. The single source/target vocabulary of this model is the union of both the source and target vocabularies. In this model, both in the encoder and decoder, all subwords are embedded in the same duo-lingual space.

4 Results

Our experiments study the quality of various embeddings, the similarity between them, and the impact of tying them on the word2vec skip-gram model, NNLMs, and NMT models.

4.1 Quality of Obtained Embeddings

In order to compare the various embeddings, we pooled five embedding evaluation methods from the literature. These evaluation methods involve calculating pairwise (cosine)

[1] http://statmt.org/wmt14/translation-task.html
[2] http://statmt.org/wmt15/translation-task.html

因此，在未采用投影正则化的模型中，每个时间步内输入词嵌入中唯一会被更新的行，就是代表当前输入词的第U_{i_t}行。这意味着表示稀有词汇的向量仅会被更新很少的次数，而输出词嵌入则会在每个时间步更新所有行。

在采用权重绑定的神经网络语言模型中，我们会设定 $U = V = S$。对于 S 中的每一行而言，其更新值即为S同时作为输入嵌入和输出嵌入时的两种角色所对应更新值之和。

第 $k \neq i_t$ 行的更新方式与未采用权重绑定的神经网络语言模型的输出嵌入中第 k 行的更新方式类似（唯一的区别在于U和V都被替换为同一个矩阵S）。在这种情况下， S的输入嵌入角色不会产生任何更新值。

第 $k = i_t$行的更新值由来自输入嵌入部分的项（情况 $k = i_t$）以及来自输出嵌入部分的项（情况 $k \neq o_t$）共同构成。第二项会随着 $p_t(i_t|i_{1:t})$呈线性增长，但由于词语很少会在同一行中出现两次（这一低概率特性也通过实验得到了验证），该值通常接近于零。因此，在这种情形下，行的更新主要受 S的输入嵌入角色所产生的更新影响。

总而言之，在采用权重绑定的神经网络语言模型中，每次迭代都会更新 S中的每一行数据；除一行之外，其余行的更新方式与未进行权重绑定的模型中输出嵌入的更新方式相似。由此可见，经过权重绑定的嵌入与未绑定模型输出嵌入的相似度，要高于其与输入嵌入的相似度。

为简化论述，上述分析主要针对神经网络语言模型展开。在word2vec模型中，更新规则类似，只不过此处 $h_2^{(t)}$ 会被恒等函数所替代。正如（Goldberg和Levy，2014）所指出的，在这种情况下并不适合采用权重绑定策略，因为如果 $p_t(i_t|i_{1:t})$的值接近于零，那么 i_t对应的嵌入向量的范数也会同样接近于零。不过这一观点并不适用于神经网络语言模型，因为LSTM层使得输入嵌入与输出嵌入之间实现了解耦。

最后，我们评估了在神经网络翻译模型中应用权重调整的效果。在该模型中：$p_t(o_t|i_{1:t}, r) = \frac{exp(V_{o_t}^\top G^{(t)})}{\sum_{x=1}^{C_t} exp(V_x^\top G^{(t)})}$其中 $r = (r_1, ..., r_N)$表示源语句中的单词集合， U 和 V 分别是解码器的输入嵌入与输出嵌入，而 W 则是编码器的输入嵌入（在翻译模型中为 $U, V \in \mathbb{R}^{C_t \times H}$ 以及 $W \in \mathbb{R}^{C_s \times H}$，此处 C_s / C_t 代表

语言对	子词 仅存在于源语言中	子词 仅存在于目标语言中	子词 在两者中
EN→FR	2K	7K	85K
EN→DE	3K	11K	80K

表1：不同语言对之间的共享BPE子词。

size of the vocabulary of the source / target). $G^{(t)}$ is the decoder, which receives the context vector, the embedding of the input word (i_t) in U, and its previous state at each timestep. c_t is the context vector at timestep t, $c_t = \sum_{j \in r} a_{tj} h_j$, where a_{tj} is the weight given to the jth annotation at time t: $a_{tj} = \frac{\exp(e_{tj})}{\sum_{k \in r} \exp(e_{ik})}$, and $e_{tj} = a_t(h_j)$, where a is the alignment model. F is the encoder which produces the sequence of annotations $(h_1, ..., h_N)$.

解码器的输出随后会通过输出嵌入 $l_t = VG^{(t)}$被投影为一个得分向量。接着，这些得分会通过softmax函数转换为概率值。

在我们的权重绑定翻译模型中，我们会将解码器的输入嵌入向量和输出嵌入向量进行绑定。

我们发现，使用BPE对ACL WMT 2014 EN→FR[1] 以及WMT 2015 EN→DE[2]数据集进行预处理时，源语言和目标语言的词汇表中出现了大量相同的子词。表1显示，英语与法语（德语）之间的BPE子词共有比例高达90%（85%）。

基于这一观察，我们提出了三向权重绑定（TWWT）方法，即将解码器的输入嵌入、解码器的输出嵌入以及编码器的输入嵌入全部进行权重绑定。该模型的单一源/目标词汇表为源语言与目标语言词汇表的并集。在该模型中，无论是编码器还是解码器，所有的子词都会被嵌入到同一个双语空间中。

4个结果

我们的实验研究了各类词嵌入的质量、它们之间的相似度，以及将这些嵌入向量与word2vec跳过语法模型、神经网络语言模型和机器翻译模型相结合所产生的影响。

4.1 所获得词嵌入的质量

为了比较不同的嵌入向量，我们整合了文献中的五种嵌入向量评估方法。这些评估方法包括计算成对（余弦）值。

1http://statmt.org/wmt14/translation-task.html
2http://statmt.org/wmt15/translation-task.html

	Input	Output	Tied
Simlex999	0.30	0.29	0.17
Verb-143	0.41	0.34	0.12
MEN	0.66	0.61	0.50
Rare-Word	0.34	0.34	0.23
MTurk-771	0.59	0.54	0.37

Table 2: Comparison of input and output embeddings learned by a word2vec skip-gram model. Results are also shown for the tied word2vec model. Spearman's correlation ρ is reported for five word embedding evaluation benchmarks.

	PTB			text8		
Embedding	In	Out	Tied	In	Out	Tied
Simlex999	0.02	0.13	0.14	0.17	0.27	0.28
Verb143	0.12	0.37	0.32	0.20	0.35	0.42
MEN	0.11	0.21	0.26	0.26	0.50	0.50
Rare-Word	0.28	0.38	0.36	0.14	0.15	0.17
MTurk771	0.17	0.28	0.30	0.26	0.48	0.45

Table 3: Comparison of the input/output embeddings of the small model from (Zaremba et al., 2014) and the embeddings from our weight tied variant. Spearman's correlation ρ is presented.

distances between embeddings and correlating these distances with human judgments of the strength of relationships between concepts. We use: Simlex999 (Hill et al., 2016), Verb-143 (Baker et al., 2014), MEN (Bruni et al., 2014), Rare-Word (Luong et al., 2013) and MTurk-771 (Halawi et al., 2012).

We begin by training both the tied and untied word2vec models on the text8[3] dataset, using a vocabulary consisting only of words that appear at least five times. As can be seen in Tab. 2, the output embedding is almost as good as the input embedding. As expected, the embedding of the tied model is not competitive. The situation is different when training the small NNLM model on either the Penn Treebank (Marcus et al., 1993) or text8 datasets (for PTB, we used the same train/validation/test set split and vocabulary as (Mikolov et al., 2011), while on text8 we used the split/vocabulary from (Mikolov et al., 2014)). These results are presented in Tab. 3. In this case, the input embedding is far inferior to the output embedding. The tied embedding is comparable to the output embedding.

A natural question given these results and the analysis in Sec. 3 is whether the word embedding in the weight tied NNLM model is more similar to the input embedding or to the output embedding

[3] http://mattmahoney.net/dc/textdata

A	B	$\rho(A,B)$ word2vec	$\rho(A,B)$ NNLM(S)	$\rho(A,B)$ NNLM(L)
In	Out	0.77	0.13	0.16
In	Tied	0.19	0.31	0.45
Out	Tied	0.39	0.65	0.77

Table 4: Spearman's rank correlation ρ of similarity values between all pairs of words evaluated for the different embeddings: input/output embeddings (of the untied model) and the embeddings of our tied model. We show the results for both the word2vec models and the small and large NNLM models from (Zaremba et al., 2014).

Model	Size	Train	Val.	Test
Large (Zaremba et al., 2014)	66M	37.8	82.2	78.4
Large + Weight Tying	51M	48.5	77.7	74.3
Large + BD (Gal, 2015) + WD	66M	24.3	78.1	75.2
Large + BD + WT	51M	28.2	75.8	73.2
RHN (Zilly et al., 2016) + BD	32M	67.4	71.2	68.5
RHN + BD + WT	24M	74.1	68.1	66.0

Table 5: Word level perplexity (lower is better) on PTB and size (number of parameters) of models that use either dropout (baseline model) or Bayesian dropout (BD). WD – weight decay.

of the original model. We, therefore, run the following experiment: First, for each embedding, we compute the cosine distances between each pair of words. We then compute Spearman's rank correlation between these vectors of distances. As can be seen in Tab. 4, the results are consistent with our analysis and the results of Tab. 2 and Tab. 3: for word2vec the input and output embeddings are similar to each other and differ from the tied embedding; for the NNLM models, the output embedding and the tied embeddings are similar, the input embedding is somewhat similar to the tied embedding, and differs considerably from the output embedding.

4.2 Neural Network Language Models

We next study the effect of tying the embeddings on the perplexity obtained by the NNLM models. Following (Zaremba et al., 2014), we study two NNLMs. The two models differ mostly in the size of the LSTM layers. In the small model, both LSTM layers contain 200 units and in the large model, both contain 1500 units. In addition, the large model uses three dropout layers, one placed right before the first LSTM layer, one between h_1 and h_2 and one right after h_2. The dropout probability is 0.65. For both the small and large models, we use the same hyperparameters (i.e. weight initialization, learning rate schedule, batch size) as in (Zaremba et al., 2014).

	输入	输出	Tied
Simlex999算法	0.30	0.29	0.17
Verb-143测试集	0.41	0.34	0.12
MEN	0.66	0.61	0.50
Rare-Word算法	0.34	0.34	0.23
MTurk-771测试集	0.59	0.54	0.37

表2：通过word2vec跳过语法模型学习得到的输入嵌入向量与输出嵌入向量的对比。同时也展示了平局word2vec模型的相关结果。针对五项词嵌入评估基准，文中给出了斯皮尔曼相关性系数 ρ 的值。

	PTB			text8		
词嵌入	In	Out	Tied	In	Out	Tied
Simlex999算法	0.02	0.13	0.14	0.17	0.27	0.28
Verb143算法	0.12	0.37	0.32	0.20	0.35	0.42
MEN	0.11	0.21	0.26	0.26	0.50	0.50
Rare-Word算法	0.28	0.38	0.36	0.14	0.15	0.17
MTurk771数据集	0.17	0.28	0.30	0.26	0.48	0.45

表3：(Zaremba等人，2014年)提出的小型模型的输入/输出嵌入向量与我们所设计的权重绑定变体模型的嵌入向量的对比。文中还给出了斯皮尔曼相关系数 ρ 的值。

比较这些嵌入向量之间的距离，并将这些距离与人类对概念间关联强度的判断结果相互对应。我们采用了以下测试集：Simlex999算法（Hill等人，2016年）、Verb-143测试集（Baker等人，2014年）、MEN算法（Bruni等人，2014年）、Rare-Word测试集（Luong等人，2013年）以及MTurk-771测试集（Halawi等人，2012年）。

我们首先在text8[3] 数据集上训练平局word2vec模型与非平局word2vec模型，所使用的词汇表仅包含出现次数至少为五次的单词。如表2所示，输出嵌入的质量几乎与输入嵌入相当。不出所料，平局模型的词嵌入效果并不理想。而当在Penn Treebank（Marcus等人，1993年）或text8数据集上训练小型神经网络语言模型时情况则有所不同（对于PTB语料库，我们采用了与（Mikolov等人，2011年）相同的训练/验证/测试集划分及词汇表；而在text8数据集上，则使用了（Mikolov等人，2014年）的划分方式与词汇表）。这些结果展示在表3中。在这种情况下，输入嵌入的质量远逊于输出嵌入，而平局模型的词嵌入则与输出嵌入水平相当。

基于这些结果以及第3节中的分析，一个自然而然的问题是：在与NNLMmodel关联的权重中使用的词嵌入，究竟更接近输入嵌入还是输出嵌入？

3http://mattmahoney.net/dc/textdata

A	B	$\rho(A,B)$ word2vec模型	$\rho(A,B)$ NNLM(S)函数	$\rho(A,B)$ NNLM(L)函数
In	Out	0.77	0.13	0.16
In	Tied	0.19	0.31	0.45
Out	Tied	0.39	0.65	0.77

表4：针对不同词嵌入向量所评估的所有词对相似度值，计算得出的斯皮尔曼等级相关系数 ρ ——这些词嵌入向量包括未进行绑定处理的模型的输入/输出嵌入向量，以及我们经过绑定处理后的模型的嵌入向量。该表展示了(Zaremba等人，2014年)研究中提出的word2vec模型以及小型和大型NNLM模型的测试结果。

模型	规模	训练集	验证集	测试集
大型（Zaremba等人，2014年）	66M	37.8	82.2	78.4
大型 + 权重绑定	51M	48.5	77.7	74.3
大型 + 边界检测算法（Gal, 2015） + 权重调整机制	66M	24.3	78.1	75.2
大型 + 边界检测算法 + 权重绑定	51M	28.2	75.8	73.2
RHN（Zilly等人，2016年）+ 边界检测算法	32M	67.4	71.2	68.5
RHN + 边界检测算法 + 权重调整机制	2400万	74.1	68.1	66.0

表5：在PTB语料库上各模型的单词级困惑度（数值越低越好），以及这些模型的规模（参数数量）；其中部分模型采用了丢弃机制（基线模型），还有模型使用了贝叶斯丢弃机制（边界检测算法）。WD表示权重衰减。

为此，我们开展了如下实验：首先针对每种嵌入向量，计算所有单词对之间的余弦距离，随后再计算这些距离向量之间的斯皮尔曼等级相关系数。如表4所示，实验结果与我们之前的分析以及表2和表3的结果一致——对于word2vec模型而言，输入嵌入和输出嵌入彼此相似，且与那种经过特殊处理的嵌入向量存在差异；而对于各类NNLM模型，输出嵌入与该类模型对应的嵌入向量较为相似，输入嵌入则仅与那种特殊处理的嵌入向量有少许相似之处，与输出嵌入的差异则相当大。

4.2 神经网络语言模型

接下来，我们研究将嵌入向量绑定在一起对神经网络语言模型所得困惑度的影响。沿袭（Zaremba等人，2014年）的研究方法，我们选择了两个神经网络语言模型进行测试。这两个模型的主要差异在于LSTM层的规模：小型模型中的两个LSTM层各包含200个单元，而大型模型中的两个LSTM层则各包含1500个单元。此外，大型模型还设置了三层丢弃机制，一层位于第一个LSTM层之前，一层位于 h_1和 h_2 之间，另一层位于 h_2之后，其丢弃概率均为0.65。对于这两种模型，我们都采用了与（Zaremba等人，2014年）研究中相同的超参数（即权重初始化方式、学习率调度策略以及批量大小）。

Model	Size	Train	Val.	Test
KN 5-gram				141
RNN				123
LSTM				117
Stack RNN	8.48M			110
FOFE-FNN				108
Noisy LSTM	4.65M		111.7	108.0
Deep RNN	6.16M			107.5
Small model	4.65M	38.0	120.7	114.5
Small + WT	2.65M	36.4	117.5	112.4
Small + PR	4.69M	50.8	116.0	111.7
Small + WT + PR	2.69M	53.5	104.9	100.9

Table 6: Word level perplexity on PTB and size for models that do not use dropout. The compared models are: KN 5-gram (Mikolov et al., 2011), RNN (Mikolov et al., 2011), LSTM (Graves, 2013), Stack / Deep RNN (Pascanu et al., 2013), FOFE-FNN (Zhang et al., 2015), Noisy LSTM (Gülçehre et al., 2016), and the small model from (Zaremba et al., 2014). The last three models are our models, which extend the small model. PR – projection regularization.

	Model	Small	S + WT	S + PR	S + WT + PR
text8	Train	90.4	95.6	92.6	95.3
	Val.	-	-	-	-
	Test	195.3	187.1	199.0	183.2
IMDB	Train	71.3	75.4	72.0	72.9
	Val.	94.1	94.6	94.0	91.2
	Test	94.3	94.8	94.4	91.5
BBC	Train	28.6	30.1	42.5	45.7
	Val.	103.6	99.4	104.9	96.4
	Test	110.8	106.8	108.7	98.9

Table 7: Word level perplexity on the text8, IMDB and BBC datasets. The last three models are our models, which extend the small model (S) of (Zaremba et al., 2014).

In addition to training our models on PTB and text8, following (Miyamoto and Cho, 2016), we also compare the performance of the NNLMs on the BBC (Greene and Cunningham, 2006) and IMDB (Maas et al., 2011) datasets, each of which we process and split into a train/validation/test split (we use the same vocabularies as (Miyamoto and Cho, 2016)).

In the first experiment, which was conducted on the PTB dataset, we compare the perplexity obtained by the large NNLM model and our version in which the input and output embeddings are tied. As can be seen in Tab. 5, weight tying significantly reduces perplexity on both the validation set and the test set, but not on the training set. This indicates less overfitting, as expected due to the reduction in the number of parameters. Recently, (Gal and Ghahramani, 2016), proposed a modified model that uses Bayesian dropout and weight decay. They obtained improved performance. When the embeddings of this model are

		Size	Validation	Test
EN→FR	Baseline	168M	29.49	33.13
	Decoder WT	122M	29.47	33.26
	TWWT	80M	29.43	33.46
EN→DE	Baseline	165M	20.96	16.79
	Decoder WT	119M	21.09	16.54
	TWWT	79M	21.02	17.15

Table 8: Size (number of parameters) and BLEU score of various translation models. TWWT – three-way weight tying.

tied, a similar amount of improvement is gained. We tried this with and without weight decay and got similar results in both cases, with slight improvement in the latter model. Finally, by replacing the LSTM with a recurrent highway network (Zilly et al., 2016), state of the art results are achieved when applying weight tying. The contribution of WT is also significant in this model.

Perplexity results are often reported separately for models with and without dropout. In Tab. 6, we report the results of the small NNLM model, that does not utilize dropout, on PTB. As can be seen, both WT and projection regularization (PR) improve the results. When combining both methods together, state of the art results are obtained. An analog table for text8, IMDB and BBC is Tab. 7, which shows a significant reduction in perplexity across these datasets when both PR and WT are used. PR does not help the large models, which employ dropout for regularization.

4.3 Neural Machine Translation

Finally, we study the impact of weight tying in attention based NMT models, using the DL4MT[4] implementation. We train our EN→FR models on the parallel corpora provided by ACL WMT 2014. We use the data as processed by (Cho et al., 2014) using the data selection method of (Axelrod et al., 2011). For EN→DE we train on data from the translation task of WMT 2015, validate on newstest2013 and test on newstest2014 and newstest2015. Following (Sennrich et al., 2016b) we learn the BPE segmentation on the union of the vocabularies that we are translating from and to (we use BPE with 89500 merge operations). All models were trained using Adadelta (Zeiler, 2012) for 300K updates, have a hidden layer size of 1000 and all embedding layers are of size 500.

Tab. 8 shows that even though the weight tied models have about 28% fewer parameters than the

[4] https://github.com/nyu-dl/dl4mt-tutorial

模型	Size	训练集	Val.	Test
KN五元语法模型				141
RNN				123
LSTM				117
堆叠循环神经网络	884.8万			110
FOFE-FNN模型				108
噪声LSTM模型	465万		111.7	108.0
深度RNN模型	616万			107.5
小型模型	465万	38.0	120.7	114.5
Small + WT	265万	36.4	117.5	112.4
Small + 投影正则化	469万	50.8	116.0	111.7
Small + WT + 投影正则化	269万	53.5	104.9	100.9

表6：未使用丢弃机制的各类模型在PTB语料库上的词级困惑度及其规模。对比的模型包括：KN五元语法模型（Mikolov等人，2011年）、RNN模型（Mikolov等人，2011年）、LSTM网络（Graves，2013年）、堆叠式深度RNN模型（Pascanu等人，2013年）、FOFE-FNN模型（张等人，2015年）、噪声LSTM模型（G¨ulc¸ehre等人，2016年），以及(Zaremba等人，2014年)提出的小型模型。最后三种模型则是我们对该小型模型进行扩展后得到的自身模型。PR——投影正则化。

	模型	Small	S + WT	S + PR	S + WT + PR
text8	训练集的困惑度分别为90.4、95.6、92.6和95.3。				
	Val.	-	-	-	-
	Test	195.3	187.1	199.0	183.2
IMDB	训练集	71.3	75.4	72.0	72.9
	Val.	94.1	94.6	94.0	91.2
	Test	94.3	94.8	94.4	91.5
BBC	训练集	28.6	30.1	42.5	45.7
	Val.	103.6	99.4	104.9	96.4
	Test	110.8	106.8	108.7	98.9

表7：在text8、IMDB以及BBC数据集上的词级困惑度。最后三种模型均为我们基于(Zaremba等人，2014年)提出的小型模型(S)进行扩展而得到的模型。

除了在PTB和text8语料库上训练我们的模型外，我们还遵循(Miyamoto和Cho，2016年)的方法，对比了这些神经网络语言模型在BBC（Greene和Cunningham，2006年）以及IMDB（Maas等人，2011年）数据集上的表现；对于这些数据集，我们都进行了处理并将其划分为训练集、验证集和测试集，同时使用的词汇表也与(Miyamoto和Cho，2016年)的研究保持一致。

在基于PTB语料库开展的首次实验中，我们对比了大型神经网络语言模型与我们对输入输出嵌入向量进行权重绑定的版本所对应的困惑度。如表5所示，权重绑定显著降低了验证集和测试集上的困惑度，但对训练集则没有影响。这一现象表明过拟合现象有所减轻，而这正是由于参数数量减少所导致的。近期，(Gal和Ghahramani, 2016)提出了一种经过改进的模型，该模型采用了贝叶斯丢弃机制和权重衰减技术，从而获得了更好的性能。当对该模型中的嵌入向量也实施权重绑定时，

		Size	验证	Test
英文→法文	基准模型	168M	29.49	33.13
	解码器WT	122M	29.47	33.26
	TWWT	80M	29.43	33.46
英文→德文	基准模型	165M	20.96	16.79
	解码器权重矩阵	119M	21.09	16.54
	TWWT	79M	21.02	17.15

表8：各类翻译模型的规模（参数数量）与BLEU分数。TWWT——三向权重绑定。

也能获得相近的性能提升效果。我们分别尝试了是否结合权重衰减的做法，发现两种情况下的结果都很相似，只是后者略好一些。最后，通过用循环高速公路网络替代LSTM网络（Zilly等人，2016），在应用权重绑定技术的条件下取得了当前最优秀的实验结果。在该模型中，权重绑定机制的作用同样十分显著。

通常会分别报告带有丢弃机制与不带丢弃机制的模型的困惑度测试结果。在表6中，我们展示了未采用丢弃机制的小型神经网络语言模型在PTB语料库上的测试结果。如图所示，无论是原始训练方法还是投影正则化都能提升模型性能；将这两种方法结合使用，则能获得当前最先进的性能表现。表7提供了text8、IMDB和BBC语料库的类似对比数据，结果显示在应用了投影正则化和原始训练方法后，这些数据集的困惑度均出现了显著下降。而对于那些已通过丢弃机制来实现正则化的大型模型而言，投影正则化则无法带来额外的提升效果。

4.3 神经机器翻译

Finally, we study the impact of weight tying in attention based NMT models, using the DL4MT[4] implementation. We train our EN→FR models on the parallel corpora provided by ACL WMT 2014. We use the data as processed by (Cho et al., 2014) using the data selection method of (Axelrod et al., 2011). For EN→DE we train on data from the translation task of WMT 2015, validate on newstest2013 and test on newstest2014 and newstest2015. Following (Sennrich et al., 2016b) we learn the BPE segmentation on the union of the vocabularies that we are translating from and to (we use BPE with 89500 merge operations). All models were trained using Adadelta (Zeiler, 2012) for 300K updates, have a hidden layer size of 1000 and all embedding layers are of size 500.

表8显示，尽管参数关联的模型其参数量比普通模型少约28%，但

4 https://github.com/nyu-dl/dl4mt-tutorial

baseline models, their performance is similar. This is also the case for the three-way weight tied models, even though they have about 52% fewer parameters than their untied counterparts.

References

Amittai Axelrod, Xiaodong He, and Jianfeng Gao. 2011. Domain adaptation via pseudo in-domain data selection. In *Proceedings of the 2011 Conference on Empirical Methods in Natural Language Processing*, pages 355–362, Edinburgh, Scotland, UK., July. Association for Computational Linguistics.

Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.

Simon Baker, Roi Reichart, and Anna Korhonen. 2014. An unsupervised model for instance level subcategorization acquisition. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 278–289. Association for Computational Linguistics.

Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. 2003. A neural probabilistic language model. *J. Mach. Learn. Res.*, 3:1137–1155, March.

Elia Bruni, Nam-Khanh Tran, and Marco Baroni. 2014. Multimodal distributional semantics. *J. Artif. Intell. Res.(JAIR)*, 49(1-47).

Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, Doha, Qatar, October. Association for Computational Linguistics.

Philip Gage. 1994. A new algorithm for data compression. *The C Users Journal*, 12(2):23–38.

Yarin Gal and Zoubin Ghahramani. 2016. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML-16)*.

Yarin Gal. 2015. A Theoretically Grounded Application of Dropout in Recurrent Neural Networks. *arXiv preprint arXiv:1512.05287*.

Yoav Goldberg and Omer Levy. 2014. word2vec explained: deriving mikolov et al.'s negative-sampling word-embedding method. *arXiv preprint arXiv:1402.3722*.

Alex Graves. 2013. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*.

Derek Greene and Pádraig Cunningham. 2006. Practical solutions to the problem of diagonal dominance in kernel document clustering. In *Proc. 23rd International Conference on Machine learning (ICML'06)*, pages 377–384. ACM Press.

Çaglar Gülçehre, Marcin Moczulski, Misha Denil, and Yoshua Bengio. 2016. Noisy activation functions. *arXiv preprint arXiv:1603.00391*.

Guy Halawi, Gideon Dror, Evgeniy Gabrilovich, and Yehuda Koren. 2012. Large-scale learning of word relatedness with constraints. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 1406–1414.

Felix Hill, Roi Reichart, and Anna Korhonen. 2016. Simlex-999: Evaluating semantic models with (genuine) similarity estimation. *Computational Linguistics*.

Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November.

Hakan Inan, Khashayar Khosravi, and Richard Socher. 2016. Tying word vectors and word classifiers: A loss framework for language modeling. *arXiv preprint arXiv:1611.01462*.

Nal Kalchbrenner and Phil Blunsom. 2013. Recurrent continuous translation models. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pages 1700–1709, Seattle, Washington, USA, October. Association for Computational Linguistics.

Thang Luong, Richard Socher, and Christopher Manning, 2013. *Proceedings of the Seventeenth Conference on Computational Natural Language Learning*, chapter Better Word Representations with Recursive Neural Networks for Morphology, pages 104–113. Association for Computational Linguistics.

L. Andrew Maas, E. Raymond Daly, T. Peter Pham, Dan Huang, Y. Andrew Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150. Association for Computational Linguistics.

Mitchell P. Marcus, Mary Ann Marcinkiewicz, and Beatrice Santorini. 1993. Building a large annotated corpus of english: The penn treebank. *Comput. Linguist.*, 19(2):313–330, June.

表8进一步表明，带权重约束的模型即便参数量较少，其性能仍能与基线模型相媲美；而三向权重约束模型虽参数量更少，约仅为未加约束模型的52%，性能也同样出色。

参考文献

Amittai Axelrod、Xiaodong He与Jianfeng Gao。2011。《通过伪领域内数据选择实现领域适配》。载于《2011年自然语言处理实证方法会议论文集》，第355–362页，英国苏格兰爱丁堡，7月。计算语言学协会。

Dzmitry Bahdanau、Kyunghyun Cho与Yoshua Bengio。2014年。《通过联合学习对齐与翻译来实现神经机器翻译》。arXIV预印本，编号arXiv:1409.0473。

Simon Baker、Roi Reichart与Anna Korhonen。2014。《一种用于实例级子类别获取的无监督模型》。载于《2014年自然语言处理实证方法会议（EMNLP）论文集》，第278–289页。计算语言学协会。

Yoshua Bengio、R´ejean Ducharme、Pascal Vincent与Christian Janvin。2003。《一种神经网络概率语言模型》。《机器学习研究杂志》，3卷：1137–1155页，3月刊。

Elia Bruni、Nam-Khanh Tran与Marco Baroni。2014。《多模态分布语义学》。《人工智能研究杂志》(JAIR)，49(1-47)。

Kyunghyun Cho、Bart van Merrienboer、Caglar Gul-cehre、Dzmitry Bahdanau、Fethi Bougares、Holger Schwenk以及Yoshua Bengio。2014年。利用RNN编码器-解码器进行投影正则化，以学习短语表示用于统计机器翻译。载于《2014年自然语言处理经验方法会议（EMNLP）论文集》，第1724–1734页，卡塔尔多哈，10月。计算语言学协会出版。

Philip Gage. 1994. 一种新的数据压缩算法.《C用户杂志》，12(2):23–38。

Yarin Gal与Zoubin Ghahramani。2016。《作为贝叶斯近似的丢弃机制：深度学习中模型不确定性的表征》。载于第33届国际机器学习大会（ICML-16）论文集。

Yarin Gal. 2015.《循环神经网络中丢弃机制的基于理论的应用》。arXIV预印本，arXiv:1512.05287。

Yoav Goldberg与Omer Levy于2014年发表了《word2vec详解：推导Mikolov等人的负采样词嵌入方法》，该论文为arXIV预印本，编号为arXiv:1402.3722。

Alex Graves在2013年提出了利用循环神经网络生成序列的方法，相关论文为arXIV预印本，编号为arXiv:1308.0850。Derek Greene与P´adraig Cunningham则在2006年针对核文档聚类中的对角优势问题提出了实用解决方案，该文发表在第23届国际机器学习大会（ICML’06）的论文集中，页码为377–384，由ACM Press出版。C¸aglar G¨ulc¸ehre、Marcin Moczulski、Misha Denil以及Yoshua Bengio于2016年发表了关于噪声激活函数的论文，对应的arXIV预印本编号为arXiv:1603.00391。

Guy Halawi、Gideon Dror、Evgeniy Gabrilovich与Yehuda Koren。2012。《在约束条件下进行大规模词关联度学习》。载于《第18届ACM SIGKDD知识发现与数据挖掘国际会议论文集》，第1406–1414页。

Felix Hill、Roi Reichart与Anna Korhonen。2016。《Simlex-999：利用（真实的）相似度估算方法评估语义模型》。《计算语言学》。

Geoffrey Hinton、Oriol Vinyals与Jeff Dean。2015年。《提炼神经网络中的知识》。arXIV预印本，编号arXiv:1503.02531。

Sepp Hochreiter与J¨urgen Schmidhuber。1997年。《长短期记忆》。《神经计算》，9(8)：1735–1780，11月刊。

Hakan Inan、Khashayar Khosravi与Richard Socher。2016年。《将词向量与词分类器相结合：一种用于语言建模的损失函数框架》。arXIV预印本，编号arXiv:1611.01462。

Nal Kalchbrenner与Phil Blunsom。2013。《循环连续翻译模型》。载于《2013年自然语言处理经验方法会议论文集》，第1700–1709页，美国华盛顿州西雅图，10月。计算语言学协会。

Thang Luong、Richard Socher与Christopher Man-nig，2013。《第十七届计算自然语言学习会议论文集》，章节“利用递归神经网络实现更优的词表示以处理形态学问题”，页码104–113。计算语言学协会。

L. Andrew Maas、E. Raymond Daly、T. Peter Pham、Dan Huang、Y. Andrew Ng以及Christopher Potts。2011。《利用词向量进行情感分析学习》。载于《计算语言学协会第49届年会：人类语言技术论文集》，第142–150页。计算语言学协会。

米切尔·P·马库斯、玛丽·安·马尔钦凯维奇与贝阿特丽斯·桑托里尼。1993年。《构建大型英语标注语料库：Penn Treebank语料库》。《计算机语言学》，19(2)：313–330，6月刊。

Tomas Mikolov, Martin Karafiát, Lukás Burget, Jan Cernocký, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *INTERSPEECH 2010, 11th Annual Conference of the International Speech Communication Association, Makuhari, Chiba, Japan, September 26-30, 2010*, pages 1045–1048.

Tomáš Mikolov, Stefan Kombrink, Lukáš Burget, Jan Černockỳ, and Sanjeev Khudanpur. 2011. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5528–5531. IEEE.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013a. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013b. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119.

Tomas Mikolov, Armand Joulin, Sumit Chopra, Michaël Mathieu, and Marc'Aurelio Ranzato. 2014. Learning longer memory in recurrent neural networks. *arXiv preprint arXiv:1412.7753*.

Bhaskar Mitra, Eric Nalisnick, Nick Craswell, and Rich Caruana. 2016. A dual embedding space model for document ranking. *arXiv preprint arXiv:1602.01137*.

Yasumasa Miyamoto and Kyunghyun Cho. 2016. Gated word-character recurrent language model. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1992–1997. Association for Computational Linguistics.

Andriy Mnih and Geoffrey E Hinton. 2009. A scalable hierarchical distributed language model. In *Advances in neural information processing systems*, pages 1081–1088.

Andriy Mnih and Koray Kavukcuoglu. 2013. Learning word embeddings efficiently with noise-contrastive estimation. In *Advances in Neural Information Processing Systems*, pages 2265–2273.

Andriy Mnih and Yee Whye Teh. 2012. A fast and simple algorithm for training neural probabilistic language models. *arXiv preprint arXiv:1206.6426*.

Razvan Pascanu, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. 2013. How to construct deep recurrent neural networks. *arXiv preprint arXiv:1312.6026*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016a. Edinburgh neural machine translation systems for wmt 16. *arXiv preprint arXiv:1606.02891*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016b. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of ACL*.

Nitish Srivastava. 2013. Improving Neural Networks with Dropout. Master's thesis, University of Toronto, Toronto, Canada, January.

Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. 2012. Lstm neural networks for language modeling. In *Interspeech*, pages 194–197, Portland, OR, USA, September.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. 2014. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*.

Matthew D Zeiler. 2012. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*.

Shiliang Zhang, Hui Jiang, Mingbin Xu, Junfeng Hou, and Li-Rong Dai. 2015. A fixed-size encoding method for variable-length sequences with its application to neural network language models. *arXiv preprint arXiv:1505.01504*.

Julian G. Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. 2016. Recurrent highway networks. *arXiv preprint arXiv:1607.03474*.

Tomas Mikolov, Martin Karafiát, Lukás Burget, Jan Cernocký, and Sanjeev Khudanpur. 2010. Recurrent neural network based language model. In *INTERSPEECH 2010, 11th Annual Conference of the International Speech Communication Association, Makuhari, Chiba, Japan, September 26-30, 2010*, pages 1045–1048.

Tomáš Mikolov, Stefan Kombrink, Lukáš Burget, Jan Černockỳ, and Sanjeev Khudanpur. 2011. Extensions of recurrent neural network language model. In *2011 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 5528–5531. IEEE.

Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013a. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013b. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119.

Tomas Mikolov, Armand Joulin, Sumit Chopra, Michaël Mathieu, and Marc'Aurelio Ranzato. 2014. Learning longer memory in recurrent neural networks. *arXiv preprint arXiv:1412.7753*.

Bhaskar Mitra, Eric Nalisnick, Nick Craswell, and Rich Caruana. 2016. A dual embedding space model for document ranking. *arXiv preprint arXiv:1602.01137*.

Yasumasa Miyamoto and Kyunghyun Cho. 2016. Gated word-character recurrent language model. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 1992–1997. Association for Computational Linguistics.

Andriy Mnih and Geoffrey E Hinton. 2009. A scalable hierarchical distributed language model. In *Advances in neural information processing systems*, pages 1081–1088.

Andriy Mnih and Koray Kavukcuoglu. 2013. Learning word embeddings efficiently with noise-contrastive estimation. In *Advances in Neural Information Processing Systems*, pages 2265–2273.

Andriy Mnih and Yee Whye Teh. 2012. A fast and simple algorithm for training neural probabilistic language models. *arXiv preprint arXiv:1206.6426*.

Razvan Pascanu, Çaglar Gülçehre, Kyunghyun Cho, and Yoshua Bengio. 2013. How to construct deep recurrent neural networks. *arXiv preprint arXiv:1312.6026*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016a. Edinburgh neural machine translation systems for wmt 16. *arXiv preprint arXiv:1606.02891*.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016b. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of ACL*.

Nitish Srivastava. 2013. Improving Neural Networks with Dropout. Master's thesis, University of Toronto, Toronto, Canada, January.

Martin Sundermeyer, Ralf Schlüter, and Hermann Ney. 2012. Lstm neural networks for language modeling. In *Interspeech*, pages 194–197, Portland, OR, USA, September.

Ilya Sutskever, Oriol Vinyals, and Quoc V Le. 2014. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112.

Wojciech Zaremba, Ilya Sutskever, and Oriol Vinyals. 2014. Recurrent neural network regularization. *arXiv preprint arXiv:1409.2329*.

Matthew D Zeiler. 2012. Adadelta: an adaptive learning rate method. *arXiv preprint arXiv:1212.5701*.

Shiliang Zhang, Hui Jiang, Mingbin Xu, Junfeng Hou, and Li-Rong Dai. 2015. A fixed-size encoding method for variable-length sequences with its application to neural network language models. *arXiv preprint arXiv:1505.01504*.

Julian G. Zilly, Rupesh Kumar Srivastava, Jan Koutník, and Jürgen Schmidhuber. 2016. Recurrent highway networks. *arXiv preprint arXiv:1607.03474*.